

02_getting_started

Video Script: Getting Started with Phase 3

Duration: 12 minutes **Difficulty:** Beginner **Goal:** Set up and create first Phase 3 workflow

[0:00-0:30] Introduction

SCRIPT: “Welcome back! In this video, we’ll get you up and running with Phase 3. We’ll cover installation, basic configuration, and create your first complete workflow using sessions, memory, and templates. By the end, you’ll have hands-on experience with all the core features.”

[0:30-2:30] Installation and Setup

[ON SCREEN: Terminal/IDE view]

SCRIPT: “First, let’s make sure you have Phase 3 properly installed. If you’re upgrading from an earlier version, the Phase 3 features are already included in HelixCode version 1.3 and later.

Let’s verify the installation:”

[Demo Commands]:

```
# Check version
helixcode --version
# Should show v1.3.0 or higher

# Verify Phase 3 modules
go list dev.helix.code/internal/session
go list dev.helix.code/internal/memory
go list dev.helix.code/internal/persistence
go list dev.helix.code/internal/template
```

SCRIPT CONTINUES: “Perfect! All Phase 3 modules are available. Now let’s look at the basic configuration.

Phase 3 uses a YAML configuration file. Here's a minimal config:"

[Show config.yaml]:

```
# Phase 3 Configuration
persistence:
  storage_path: "./helixcode_data"
  format: "json"
  auto_save: true
  auto_save_interval: 300 # 5 minutes

session:
  max_history: 100
  default_mode: "building"

memory:
  max_messages_per_conversation: 1000
  max_total_tokens: 100000

templates:
  template_directory: "./templates"
  load_builtin: true
```

SCRIPT CONTINUES: "Let's break this down:

Persistence - where to save data, what format, and auto-save settings. I recommend starting with auto-save enabled every 5 minutes.

Session - how many historical sessions to keep and the default mode.

Memory - limits for conversations to prevent unbounded growth.

Templates - where to store custom templates and whether to load built-in ones.

Save this as `helixcode-config.yaml` in your project root."

[2:30-5:00] Initializing the System

[ON SCREEN: Code editor]

SCRIPT: "Now let's write code to initialize all Phase 3 systems. Create a file called `main.go`:"

[Show code with syntax highlighting]:

```

package main

import (
    "fmt"
    "dev.helix.code/internal/session"
    "dev.helix.code/internal/memory"
    "dev.helix.code/internal/persistence"
    "dev.helix.code/internal/template"
)

func main() {
    // Initialize all managers
    sessionMgr := session.NewManager()
    memoryMgr := memory.NewManager()
    templateMgr := template.NewManager()

    // Initialize persistence store
    store := persistence.NewStore("./helixcode_data")
    store.SetSessionManager(sessionMgr)
    store.SetMemoryManager(memoryMgr)
    store.SetTemplateManager(templateMgr)

    // Load built-in templates
    if err := templateMgr.RegisterBuiltinTemplates(); err != nil
    {
        fmt.Printf("Error loading templates: %v\n", err)
        return
    }

    // Enable auto-save every 5 minutes
    store.EnableAutoSave(300)

    // Try to restore previous state
    if err := store.Load(); err != nil {
        fmt.Println("No previous state found, starting fresh")
    } else {
        fmt.Println("Restored previous state successfully")
    }

    fmt.Println("HelixCode Phase 3 initialized!")
    fmt.Printf("Sessions: %d\n", sessionMgr.Count())
    fmt.Printf("Templates: %d\n", templateMgr.Count())
}

```

SCRIPT CONTINUES: “This is your basic initialization pattern. We create managers for each system, set up the persistence store, load built-in templates, enable auto-save, and attempt to restore any previous state.

Notice how simple this is - just a few lines and you have a complete Phase 3 environment.

Let’s run it:”

[Demo: Run command]:

```
go run main.go
# Output:
# No previous state found, starting fresh
# HelixCode Phase 3 initialized!
# Sessions: 0
# Templates: 5
```

SCRIPT CONTINUES: “Great! We have 5 built-in templates ready to use, and the system is initialized.”

[5:00-8:00] Creating Your First Workflow

SCRIPT: “Now for the fun part - let’s create a complete workflow. We’ll implement a simple feature using all Phase 3 systems.

Add this to your main.go:”

[Show code]:

```
func implementFeature() {
    // 1. Create a development session
    sess := sessionMgr.Create(
        "add-user-auth",
        session.ModeBuilding,
        "api-server",
    )
    sess.AddTag("authentication")
    sess.AddTag("security")

    // Start the session
    if err := sessionMgr.Start(sess.ID); err != nil {
        fmt.Printf("Error starting session: %v\n", err)
        return
    }
}
```

```

fmt.Printf("Started session: %s\n", sess.Name)
fmt.Printf("Mode: %s, Status: %s\n", sess.Mode, sess.Status)

// 2. Create a conversation for this work
conv := memoryMgr.CreateConversation("User Authentication
    Implementation")
conv.SessionID = sess.ID
conv.SetMetadata("feature", "user-auth")

// 3. Add messages to build context
memoryMgr.AddMessage(conv.ID, memory.NewSystemMessage(
    "You are helping implement user authentication for an API
    server.",
))

memoryMgr.AddMessage(conv.ID, memory.NewUserMessage(
    "I need to implement JWT-based authentication. What's the
    best approach?",
))

// Simulate AI response
memoryMgr.AddMessage(conv.ID, memory.NewAssistantMessage(
    "For JWT authentication in Go, I recommend: 1) Use
    golang-jwt library...",
))

fmt.Printf("Conversation started with %d messages\n",
    len(conv.GetMessages()))

// 4. Use a template to generate code
funcCode, err := templateMgr.RenderByName("Function",
    map[string]interface{}{
        "function_name": "GenerateJWT",
        "parameters":    "userID string, expiresIn
        time.Duration",
        "return_type":   "(string, error)",
        "body":          "// Implementation here\nreturn \"\",
        nil",
    })

if err != nil {
    fmt.Printf("Template error: %v\n", err)
}

```

```

        return
    }

    fmt.Println("\nGenerated code:")
    fmt.Println(funcCode)

    // 5. Save the code as a message
    memoryMgr.AddMessage(conv.ID, memory.NewAssistantMessage(
        fmt.Sprintf("Here's the function:\n\n``go\n%s\n``",
            funcCode),
    ))

    // 6. State is auto-saved, but we can manually save too
    if err := store.Save(); err != nil {
        fmt.Printf("Save error: %v\n", err)
        return
    }

    fmt.Println("\nWorkflow complete! State saved.")

    // Show session statistics
    stats := sessionMgr.GetStatistics()
    fmt.Printf("\nSession Statistics:\n")
    fmt.Printf("    Total sessions: %d\n", stats.TotalSessions)
    fmt.Printf("    Active sessions: %d\n", stats.ActiveSessions)
}

```

SCRIPT CONTINUES: “Let’s walk through what this does:

Step 1 - Create a session called ‘add-user-auth’ in Building mode for the ‘api-server’ project. We tag it for easy filtering later.

Step 2 - Create a conversation associated with this session. The conversation will hold all our AI interactions.

Step 3 - Add messages to build context. A system message sets the AI’s role, and user/assistant messages capture the conversation.

Step 4 - Use the built-in Function template to generate code. We provide variables like function name, parameters, and return type.

Step 5 - Save the generated code back to the conversation for reference.

Step 6 - Manually save state. Remember, auto-save is also running in the background.

Let's run this:"

[Demo: Run]:

```
go run main.go
```

[Show output]:

```
Started session: add-user-auth  
Mode: building, Status: active  
Conversation started with 3 messages
```

Generated code:

```
func GenerateJWT(userID string, expiresIn time.Duration) (string, error) {  
    // Implementation here  
    return "", nil  
}
```

Workflow complete! State saved.

Session Statistics:

```
Total sessions: 1  
Active sessions: 1
```

[8:00-10:00] Persistence in Action

SCRIPT: "Now here's the magic - let's close the program and restart to see persistence in action."

Stop the program with Ctrl+C, then run it again:"

[Demo: Restart]:

```
go run main.go
```

[Show output]:

```
Restored previous state successfully  
HelixCode Phase 3 initialized!  
Sessions: 1  
Templates: 5
```

SCRIPT CONTINUES: "Notice 'Restored previous state successfully' and we have 1 session. Let's add code to verify our data is intact:"

[Show code]:

```
// Add to main()
func verifyRestore() {
    sessions := sessionMgr.GetAll()
    fmt.Printf("\nRestored sessions:\n")
    for _, s := range sessions {
        fmt.Printf("  - %s (Mode: %s, Status: %s)\n",
            s.Name, s.Mode, s.Status)
    }

    convs := memoryMgr.GetAll()
    fmt.Printf("\nRestored conversations:\n")
    for _, c := range convs {
        fmt.Printf("  - %s (%d messages)\n",
            c.Title, len(c.GetMessages()))
    }
}
```

[Demo: Run with verification]:

Restored sessions:

- add-user-auth (Mode: building, Status: active)

Restored conversations:

- User Authentication Implementation (5 messages)

SCRIPT CONTINUES: “Perfect! Our session and conversation are exactly as we left them. This is what Phase 3 persistence gives you - complete state recovery.”

[10:00-11:30] Quick Wins and Best Practices

SCRIPT: “Before we wrap up, here are some quick wins and best practices:

1. **Use descriptive session names** - ‘add-user-auth’ is better than ‘session1’
2. **Tag your sessions** - Makes filtering and searching much easier later
3. **Associate conversations with sessions** - Maintains the relationship between work and context
4. **Let auto-save work** - Don’t disable it unless you have a specific reason
5. **Use built-in templates** - They’re production-ready and cover common scenarios
6. **Set appropriate limits** - Prevent conversations from growing unbounded

7. Periodically trim old data - Use the trimming features to manage size

8. Export important work - Sessions and templates can be exported for backup or sharing”

[Visual: Show each tip as an overlay]

[11:30-12:00] Conclusion

SCRIPT: “Congratulations! You’ve just: - Set up Phase 3 with proper configuration - Initialized all four core systems - Created a complete development workflow - Seen state persistence in action - Learned best practices for success

In the next videos, we’ll dive deep into each feature, exploring advanced capabilities and real-world patterns.

Ready to master sessions? See you in the next video!”

[ON SCREEN: “Next: Session Management Fundamentals”]

Supplementary Materials

Complete Code Example

See `/examples/getting-started/main.go` for the full working example.

Configuration Template

```
persistence:
  storage_path: "./helixcode_data"
  format: "json"
  auto_save: true
  auto_save_interval: 300

session:
  max_history: 100
  default_mode: "building"

memory:
  max_messages_per_conversation: 1000
  max_total_tokens: 100000

templates:
```

```
template_directory: "../templates"  
load_builtin: true
```

Key Takeaways

1. Installation is simple - included in HelixCode v1.3+
2. Configuration uses YAML for easy customization
3. Initialize all managers and connect them to persistence
4. Basic workflow: Create session → Start conversation → Use templates → Auto-save
5. State restoration is automatic on restart
6. Built-in templates provide instant productivity

Quiz

1. What version of HelixCode includes Phase 3?
2. What format does Phase 3 configuration use?
3. How do you enable auto-save?
4. What happens when you restart after saving state?
5. How many built-in templates are provided?